Tarefa 16: Produto Matriz-Vetor MPI

Escondendo Latência com Sobreposição de Computação

Werbert Arles de Souza Barradas

DCA3703 - Programação Paralela
UFRN - Universidade Federal do Rio Grande do Norte

Setembro de 2025

Introdução: O Problema

O objetivo deste trabalho é analisar a performance de uma implementação paralela para o cálculo do produto matriz-vetor, utilizando a biblioteca MPI (Message Passing Interface).

A operação matemática central é:

$$y = A \cdot x$$

Onde:

- A é uma matriz de dimensões $M \times N$.
- x é um vetor de tamanho N.
- y é o vetor resultante de tamanho M.

Objetivo: Avaliar como o tempo de execução varia com o aumento do tamanho da matriz e do número de processos.

Estratégia de Paralelização com MPI

A solução foi implementada seguindo uma estratégia de paralelismo de dados:

1. **Distribuição da Matriz (Scatter):**
   - A matriz A é dividida em blocos de linhas.
   - O processo 0 distribui um bloco de linhas para cada processo (incluindo ele mesmo) usando a função `MPI_Scatter`.
2. **Distribuição do Vetor (Broadcast):**
   - O vetor x completo é necessário por todos os processos.
   - O processo 0 envia uma cópia de x para todos os outros processos com a função `MPI_Bcast`.
3. **Cálculo Local e Agrupamento (Gather):**
   - Cada processo calcula uma parte do vetor de resultado y.
   - Os resultados parciais são enviados de volta para o processo 0, que os monta na ordem correta usando `MPI_Gather`.

Configuração do Benchmark

O experimento foi conduzido para avaliar a escalabilidade da solução.

Parâmetros Variados:

- **Tamanho da Matriz (M=N):** 1000, 2000, 4000, 6000, 8000, e 10000.
- **Número de Processos (P):** 2, 4, e 8.

Métricas Coletadas:

- **Tempo de Execução (s):** Tempo total gasto (cálculo + comunicação).
- **Speedup:** Ganho de performance em relação à execução com 2 processos.
- **Eficiência:** Medida de quão bem os recursos dos processadores foram aproveitados.

Resultados: Tabela de Dados (Parte 1)

Linhas(M)	Colunas(N)	Proc(P)	Tempo(s)	Speedup	Eficiência
1000	1000	2	0,006713	1,000000	0,500000
1000	1000	4	0,006134	1,094392	0,273598
1000	1000	8	0,002652	2,531297	0,316412
2000	2000	2	0,024679	1,000000	0,500000
2000	2000	4	0,024429	1,010234	0,252558
2000	2000	8	0,026069	0,946680	0,118335
4000	4000	2	0,102049	1,000000	0,500000
4000	4000	4	0,096437	1,058193	0,264548
4000	4000	8	0,092085	1,108204	0,138526

Resultados: Tabela de Dados (Parte 2)

Linhas(M)	Colunas(N)	Proc(P)	Tempo(s)	Speedup	Eficiência
6000	6000	2	0,221315	1,000000	0,500000
6000	6000	4	0,218902	1,011023	0,252756
6000	6000	8	0,210088	1,053440	0,131680
8000	8000	2	0,459333	1,000000	0,500000
8000	8000	4	0,429323	1,069901	0,267475
8000	8000	8	0,389038	1,180689	0,147586
10000	10000	2	0,696918	1,000000	0,500000
10000	10000	4	0,910461	0,765456	0,191364
10000	10000	8	1,049443	0,664084	0,083010

Resultados: Gráfico de Tempo de Execução

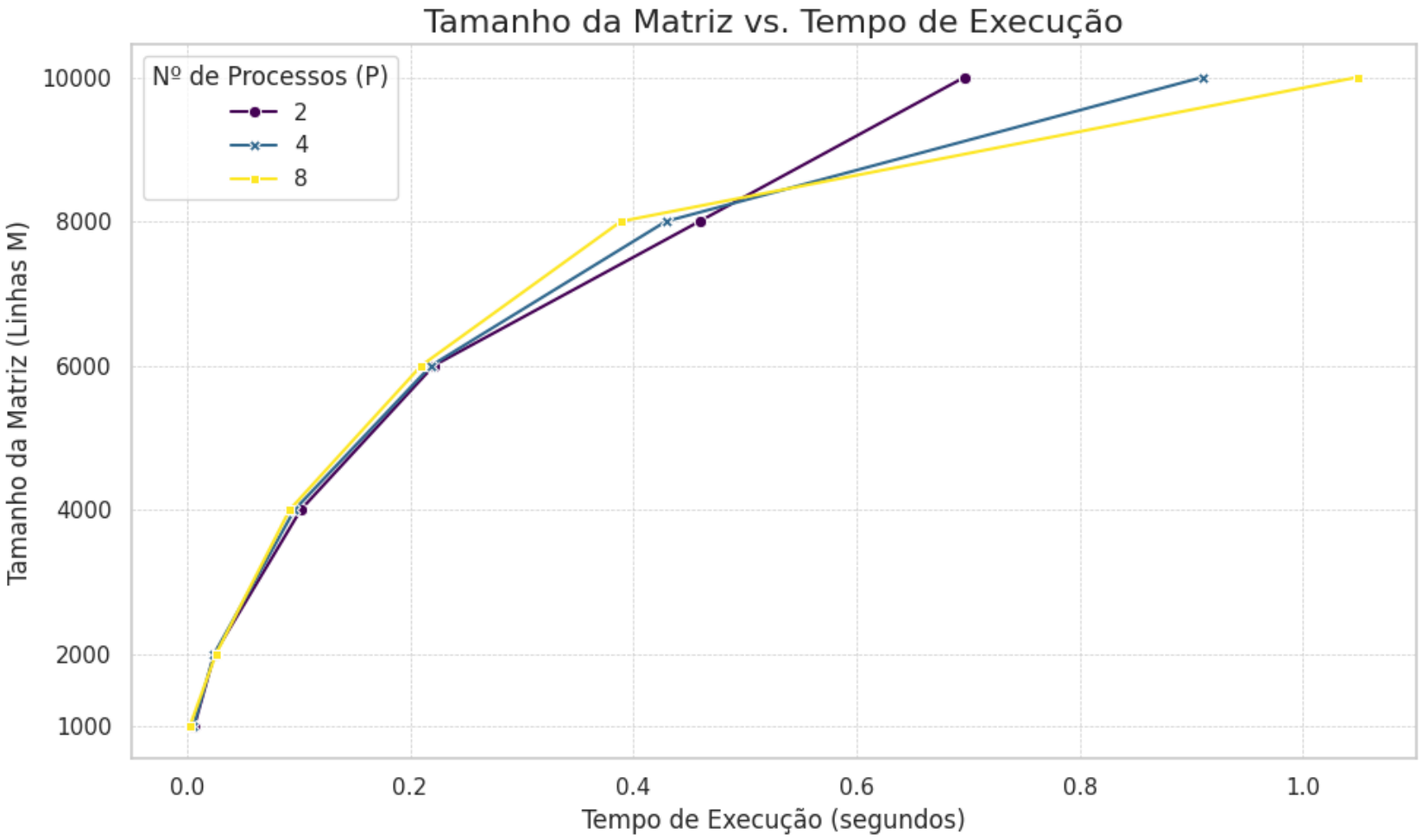

Figura: Comparativo do tempo de execução para cada tamanho de matriz

Análise dos Resultados

- **Tempo de Execução:** Para a maioria dos casos, o aumento do número de processos levou a uma redução no tempo de execução, como esperado. No entanto, para a matriz 10000x10000, o tempo aumentou, sugerindo um gargalo de comunicação.
- **Gargalo:** A operação `MPI_Bcast` (enviando o vetor inteiro para todos) e a `MPI_Gather` são os principais pontos de *overhead*, limitando a escalabilidade.

Conclusão

- A paralelização do produto matriz-vetor com MPI foi implementada com sucesso, seguindo o modelo de divisão de dados por linhas.
- Os resultados mostram que, embora a paralelização reduza o tempo de execução, a escalabilidade é limitada pelo *overhead* de comunicação, um fator crítico em algoritmos distribuídos.
- Para problemas muito grandes (matriz 10000x10000), o custo de comunicação superou o ganho do cálculo paralelo, resultando em um *slowdown*.

Matrix_vetor_limpo.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <time.h>

// Protótipo da função de inicialização
void initialize_matrix_and_vector(double *A, double *x, int M, int N);

int main(int argc, char *argv[]) {
    int rank, size;
    double start_time, end_time;

    // 1. Inicialização do ambiente MPI
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 3) {
        if (rank == 0) {
            fprintf(stderr, "Uso: %s <M_linhas> <N_colunas>\n", argv[0]);
        }
        MPI_Finalize();
        return 1;
    }

    int M_GLOBAL = atoi(argv[1]);
    int N_GLOBAL = atoi(argv[2]);
    int M = M_GLOBAL;
    int N = N_GLOBAL;

    // Para um Scatter simples, M deve ser divisível pelo número de processos.
    // O código ajusta M para o maior múltiplo de 'size' menor que o M original.
    if (M % size != 0) {
        if (rank == 0) {
            fprintf(stderr, "AVISO: M (%d) não é divisível por %d. Ajustando M para %d.\n", M, size, M - (M % size));
        }
        M -= (M % size);
    }

    // O número de linhas que cada processo irá gerenciar
    int local_M = M / size;

    // Alocação de memória
    double *A_global = NULL; // Matriz completa (somente no rank 0)
    double *y_global = NULL; // Vetor resultado (somente no rank 0)
    double *x = (double *)malloc(N * sizeof(double)); // Vetor x (em todos os processos)
    double *A_local = (double *)malloc(local_M * N * sizeof(double));  // Bloco local da matriz A
```

```c
    double *y_local = (double *)malloc(local_M * sizeof(double));  // Bloco local do resultado y

    // O processo raiz (rank 0) aloca e inicializa os dados completos
    if (rank == 0) {
        A_global = (double *)malloc(M * N * sizeof(double));
        y_global = (double *)malloc(M * sizeof(double));
        initialize_matrix_and_vector(A_global, x, M, N);

        // Início da contagem de tempo
        start_time = MPI_Wtime();
    }

    // 2. Distribuição de Dados
    // Envia o vetor 'x' completo para todos os processos
    MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    // Distribui as linhas da matriz 'A_global' entre todos os processos
    MPI_Scatter(A_global, local_M * N, MPI_DOUBLE,
                A_local, local_M * N, MPI_DOUBLE,
                0, MPI_COMM_WORLD);

    // Sincroniza todos os processos antes da computação
    MPI_Barrier(MPI_COMM_WORLD);

    // 3. Computação Local
    // Cada processo calcula o produto para sua porção local da matriz
    for (int i = 0; i < local_M; i++) {
        y_local[i] = 0.0;
        for (int j = 0; j < N; j++) {
            y_local[i] += A_local[i * N + j] * x[j];
        }
    }

    // 4. Agregação de Resultados
    // Reúne os resultados locais 'y_local' de cada processo no vetor 'y_global' do processo raiz
    MPI_Gather(y_local, local_M, MPI_DOUBLE,
               y_global, local_M, MPI_DOUBLE,
               0, MPI_COMM_WORLD);

    // 5. Finalização e Saída
    // O processo raiz finaliza a contagem de tempo e exibe os resultados
    if (rank == 0) {
        end_time = MPI_Wtime();
        double elapsed_time = end_time - start_time;

        printf("--- Produto Matriz-Vetor Distribuído ---\n");
        printf("Tamanho da Matriz A: %d x %d\n", M_GLOBAL, N_GLOBAL);
        printf("Número de Processos: %d\n", size);
        printf("Linhas por Processo (M/size): %d\n", local_M);
        printf("Tempo de Execução (Computação + Comunicação): %.6f segundos\n", elapsed_time);
```

```c
    }

    // Liberação de memória
    free(A_local);
    free(y_local);
    free(x);
    if (rank == 0) {
        free(A_global);
        free(y_global);
    }

    MPI_Finalize();
    return 0;
}

// Função para inicializar a matriz A e o vetor x com valores determinísticos
void initialize_matrix_and_vector(double *A, double *x, int M, int N) {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            A[i * N + j] = (double)(i % 10 + j % 10) / 10.0;
        }
    }
    for (int j = 0; j < N; j++) {
        x[j] = 1.0;
    }
}
```